Plano de Implementação: POC Acadêmica de Triagem Inicial e Agendamento

1. Visão Geral e Alinhamento de Escopo

Este documento detalha o plano de desenvolvimento da **POC acadêmica de triagem inicial e agendamento**, estruturada conforme os requisitos e critérios de aceite definidos no arquivo de especificação.

Objetivos Principais

- **Demonstrar redução do retrabalho** na coleta preliminar de informações clínicas por meio de fluxo digital guiado.
- **Garantir validação humana obrigatória** (triagem pelo enfermeiro) antes de liberar o agendamento de consultas.
- **Organização determinística de agenda** com bloqueio transacional e constraints contra reservas duplicadas e sobreposição de horários do paciente.
- **Compartilhamento interinstitucional controlado** entre duas instituições fictícias (*Clínica Santa Maria* e *Clínica São Bento*), baseado em consentimento explícito e revogável do paciente.
- **Ciclo pós-atendimento integrado** com geração atômica de notificações simuladas (WhatsApp/E-mail), pesquisa de satisfação e dashboard operacional.

Limites e Restrições Explícitas (Não-Escopo)

- **Zero IA / Zero LLM**: O chatbot é uma máquina de estados finitos determinística com respostas estruturadas e templates de texto fixos. Não serão integrados Ollama, OpenAI, NLP ou bancos vetoriais.
- **Comunicação Simulada**: WhatsApp e E-mail são puramente simulados no banco de dados e exibidos em uma central de notificações interna da SPA. Sem provedores externos, webhooks, chips reais ou SMTP.
- **Execução 100% Offline / Local**: O sistema deve rodar sem conexão à internet externa após instalação das dependências.
- **Monólito Modular**: Uma única Web API ASP.NET Core (.NET 8 LTS) e uma única SPA Angular com navegação orientada a perfis (sem microserviços nem múltiplos apps).

2. Arquitetura da Solução

Tecnologias e Versões Alvo

- **Backend**: ASP.NET Core 8 Web API (.NET 8.0 LTS), Entity Framework Core 8, Npgsql, MongoDB.Driver 2.x/3.x, ASP.NET Core Identity com JWT.
- **Frontend**: Angular (v17/v18) com TypeScript, Angular Router, Reactive Forms, interface limpa e responsiva simulando layout de WhatsApp no chat.
- **Bancos de Dados**:
  - **PostgreSQL**: Fonte de verdade relacional (tabelas operacionais, constraints parciais, UUIDs v4, datas em UTC).
  - **MongoDB**: Armazenamento flexível das sessões conversacionais brutas (`conversas`).
- **Armazenamento de Anexos**: Diretório local com subpastas organizadas por hash/UUID, com validação de extensão, tamanho (máx 5MB) e assinaturas binárias (magic bytes).

3. Matriz de Perfis e Permissões (RBAC)

A segurança deve ser validada **rigorosamente no backend** em todas as requisições, inclusive parâmetros de rota e downloads:

Perfil	Ações Permitidas	Restrições Rígidas no Backend
Paciente	Acessar e editar seus próprios dados; interagir no chat; visualizar slots e reservar suas consultas; ver seus documentos/evoluções; gerenciar autorizações de compartilhamento; responder pesquisa de satisfação; acessar caixa simulada.	Bloqueado para dados de qualquer outro paciente (retorna 403 mesmo alterando UUIDs na rota); não agenda sem triagem validada.
Atendente	Cadastro operacional de pacientes; gestão de slots da agenda da sua instituição; agendamento e cancelamento de consultas; acesso ao Dashboard operacional consolidado.	**Sem acesso a dados clínicos**: proibido de acessar mensagens da coleta, resumo preliminar/validado, evoluções médicas e download de exames.
Enfermeiro	Visualizar fila de coletas pendentes da sua instituição; ler histórico do chat e resumo preliminar; corrigir/editar campos da coleta; validar coleta ou encaminhar para presencial.	Proibido de registrar evolução médica de consulta e de gerenciar usuários/administração.
Médico	Listar suas consultas agendadas/em atendimento; visualizar histórico clínico e documentos de pacientes vinculados; registrar evolução textual e concluir atendimento.	Não tem acesso irrestrito a todos os pacientes da clínica; só visualiza prontuário de pacientes com consulta marcada ou atendida por ele. Acesso interinstitucional exige autorização ativa do paciente.
Admin Técnico	Gerenciar contas de usuários e vínculos institucionais; visualizar status de saúde dos serviços (`/api/health`).	Sem acesso clínico a prontuários, sintomas, chats e arquivos anexos.

4. Estrutura do Modelo de Dados

4.1. PostgreSQL (Relacional & Transacional)

Todas as chaves primárias utilizam `Guid` (`uuid`), e os campos temporais são padronizados em `DateTime` UTC (`timestamptz`).

- `Usuarios` (AspNetUsers customizado): Id, Nome, Email, Perfil (`Enum`), InstituicaoId (opcional).
- `Instituicoes` : Id, Nome, RazaoSocial, Ativa.
- `Pacientes` : Id, UsuarioId (FK única 1:1), Nome, DataNascimento, TelefoneFicticio, DocumentoFicticio.
- `Medicos` : Id, UsuarioId (FK única 1:1), InstituicaoId (FK), Especialidade, CrmFicticio.

- **`SlotsAgenda`**: Id, MedicoId (FK), InstituicaoId (FK), InicioUtc, FimUtc (slots de 30 min).
- **`ColetasIniciais`**: Id, PacienteId (FK), InstituicaoId (FK), Status (`EmColeta`, `PendenteValidacao`, `Validada`, `EncaminhadaPresencial`), ConversaId (UUID lógico para o MongoDB), ResumoValidado, EnfermeiroId (FK nula até validação), ValidadoEmUtc.
- **`Consultas`**: Id, PacienteId (FK), SlotId (FK), ColetaId (FK), Status (`Agendada`, `EmAtendimento`, `Concluida`, `Cancelada`), CriadaEmUtc.
  - *Constraint única parcial*: `CREATE UNIQUE INDEX uq_consultas_slot_ativo ON Consultas (SlotId) WHERE Status != 'Cancelada';`
- **`Evolucoes`**: Id, ConsultaId (FK única 1:1), MedicoId (FK), Texto, CriadaEmUtc.
- **`DocumentosPaciente`**: Id, PacienteId (FK), InstituicaoOrigemId (FK), NomeOriginal, Extensao, MimeType, TamanhoBytes, ChaveArquivo, CriadoEmUtc.
- **`AutorizacoesCompartilhamento`**: Id, PacienteId (FK), InstituicaoOrigemId (FK), InstituicaoDestinoId (FK), CriadaEmUtc, RevogadaEmUtc (nulo se ativa).
- **`PesquisasSatisfacao`**: Id, ConsultaId (FK única 1:1), PacienteId (FK), Nota (1-5), Percepcao (`Melhor`, `Igual`, `Pior`, `NaoResponder`), Comentario, RespondidaEmUtc.
- **`NotificacoesSimuladas`**: Id, PacienteId (FK), ConsultaId (FK), Canal (`WhatsApp`, `Email`), Finalidade (`ConvitePesquisa`, `LembreteConsulta`), Titulo, Mensagem, CriadaEmUtc, LidaEmUtc.

4.2. MongoDB (Conversas e Resumos Preliminares)

- **Coleção `conversas`**:

```
json
{
  "_id": "uuid-da-conversa",
  "coletaId": "uuid-da-coleta-postgres",
  "pacienteId": "uuid-do-paciente",
  "instituicaoId": "uuid-da-instituicao",
  "etapaAtual": 3,
  "versaoRoteiro": 1,
  "status": "EmAndamento",
  "resumoPreliminar": {
    "queixa": "Dor de cabeça persistente",
    "duracao": "3 dias",
    "alergias": "Dipirona",
    "medicamentos": "Paracetamol 500mg",
    "observacoes": "Piora com claridade"
  },
  "mensagens": [
    {
      "id": "uuid-msg-1",
      "remetente": "Bot",
      "conteudo": "Olá! Vamos iniciar sua coleta guiada...",
      "tipo": "Texto",
      "opcoes": null,
      "enviadaEmUtc": "2026-10-04T00:00:00Z"
    },
    {
      "id": "uuid-msg-2",
      "remetente": "Paciente",
      "conteudo": "Dor de cabeça persistente",
      "tipo": "Texto",
      "clienteMessageId": "client-uuid-1",
      "enviadaEmUtc": "2026-10-04T00:01:00Z"
    }
  ],
  "criadaEmUtc": "2026-10-04T00:00:00Z",
  "atualizadaEmUtc": "2026-10-04T00:05:00Z"
}
```

5. Máquina de Estados do Chatbot Guiado (Sem IA)

Regras do Motor Determinístico:

1. **Idempotência**: Cada mensagem enviada pelo frontend contém `clienteMessageId` e o número da `etapaEsperada`. Requisições duplicadas não avançam etapa.
2. **Entrada Fora do Roteiro**: Se o paciente enviar comandos aleatórios ou perguntas abertas, o backend devolve a resposta padrão obrigatória:

   > "*Nesta demonstração, consigo coletar informações para a equipe. Para outras dúvidas, solicite atendimento humano.*"

3. **Template de Resumo Automático**:

   > "*Queixa relatada: {queixa}. Duração relatada: {duracao}. Alergias relatadas: {alergias}. Medicamentos relatados: {medicamentos}. Observações: {observacoes}.*" Campos em branco recebem `"não informado"`.

6. Fases de Implementação Incremental

A execução será dividida em 6 fases lógicas, garantindo testes e critérios de aceite a cada entrega:

Fase 1: Fundação, Infraestrutura, Identity e Seeds

- Criar estrutura do repositório:
  - `backend/` (.NET 8 Web API, padrão Clean/Modular Monolith com DTOs explícitos).
  - `frontend/` (Angular com roteador por perfis e Tailwind/Componentes).
  - `docker-compose.yml` para instanciar PostgreSQL 16 e MongoDB 7 com volumes persistentes.
  - `.env.example` e `appsettings.json` com strings de conexão locais.
- Configurar EF Core com PostgreSQL (`ApplicationDbContext`), Migrations e Npgsql.
- Configurar conexão com MongoDB via driver oficial C#.
- Implementar autenticação via ASP.NET Core Identity e geração de Bearer JWT.
- Implementar serviço de Seed idempotente com:
  - 2 Instituições: *Clínica Central* e *Clínica Esperança*.
  - 2 Médicos (1 por clínica, com especialidades distintas).

    - 2 Enfermeiros (1 por clínica).
    - 2 Atendentes (1 por clínica).
    - 2 Pacientes fictícios.
    - 1 Admin técnico.
    - Slots de agenda gerados dinamicamente para os próximos 7 dias.
- No Frontend: Tela de Login moderna contendo um **"Seletor Rápido de Contas de Demonstração"** para alternar facilmente de perfil durante apresentações acadêmicas.

Fase 2: Gestão de Slots de Agenda e Controle Estrito de Concorrência

- Endpoints de consulta de slots disponíveis (`GET /api/slots?medicoId=&data=`).
- Endpoint de agendamento (`POST /api/consultas`):
    - Validação de que o paciente possui Coleta no status `Validada` .
    - Transação relacional do PostgreSQL com row-level lock (`SELECT ... FOR UPDATE` na linha do paciente) para impedir que o mesmo paciente agende duas consultas com horários concorrentes.
    - Constraint de índice parcial no PostgreSQL impedindo reserva dupla do mesmo slot.
    - Tratamento de exceção de concorrência com retorno explícito **HTTP 409 Conflict**.
- No Frontend: Telas de calendário e seleção de horários para o Paciente e para o Atendente (agendamento em nome do paciente).

Fase 3: Chatbot Determinístico e Fila de Validação do Enfermeiro

- Endpoint de inicialização da coleta (`POST /api/coletas`): cria coleta no PostgreSQL e documento no MongoDB de forma idempotente.
- Endpoints do motor de chat (`POST /api/coletas/{id}/mensagens` e `POST /api/coletas/{id}/finalizacao`):
    - Máquina de estados determinística em 8 passos.
    - Suporte à retomada de sessão (refresh de página reidrata o estado salvo no MongoDB).
    - Montagem automática do resumo preliminar baseado no template.
- No Frontend: Interface de Chat com design visual inspirado em WhatsApp, com aviso obrigatório: "*Canal simulado — coleta guiada sem IA*".
- Fila de Triagem do Enfermeiro (`GET /api/coletas?status=PendenteValidacao` e `PUT /api/coletas/{id}/validacao`):
    - Visualização da transcrição completa do chat (MongoDB).

    - Formulário para edição/correção dos dados coletados.
    - Ações: **Validar** (libera agendamento e salva resumo oficial no PostgreSQL) ou **Encaminhar para Presencial** (bloqueia agendamento automático).

Fase 4: Prontuário, Evolução Médica, Anexos e Compartilhamento Cross-Institution

- Visão do Médico: lista de consultas da sua instituição e atendimento da consulta (`PUT /api/consultas/{id}/evolucao` e `POST /api/consultas/{id}/conclusao`).
    - Conclusão exige texto de evolução obrigatório.
- Sistema de Anexos (`POST /api/pacientes/{id}/documentos` e `GET /api/documentos/{id}/arquivo`):
    - Limite de 5MB, validação de extensão e magic bytes.
    - Armazenamento em disco com UUID interno.
    - Download protegido: o backend valida se quem está baixando é o próprio paciente ou um médico com atendimento autorizado.
- Compartilhamento Interinstitucional:
    - Paciente autoriza compartilhamento da Clínica A para Clínica B (`POST /api/pacientes/{id}/autorizacoes`).
    - Médico da Clínica B com consulta vinculada consegue ler histórico e anexos originados em A.
    - Paciente revoga (`DELETE /api/autorizacoes/{id}`); a partir desse momento, médico de B recebe HTTP 403 para novos acessos aos dados de A.

Fase 5: Notificações Simuladas, Pesquisa de Satisfação e Dashboard Operacional

- Ao concluir a consulta médica:
    - Criação transacional atômica no PostgreSQL do registro pendente de `PesquisaSatisfacao` .
    - Criação de 2 registros em `NotificacoesSimuladas` (um simulando WhatsApp e um simulando E-mail com o link interno da pesquisa).
- Central de Notificações na SPA (abas "WhatsApp Simulado" e "E-mail Simulado").
- Formulário da Pesquisa: Nota (1 a 5), percepção e comentário. Validação de resposta única.
- Dashboard Operacional do Atendente (`GET /api/dashboard`):
    - Total de consultas por status (Agendadas, Em Atendimento, Concluídas, Canceladas).
    - Coletas pendentes de validação.

    - Média de satisfação considerando estritamente respostas enviadas.

Fase 6: Testes Automatizados de Integração e Documentação Técnica

- Criar projeto de testes com `xUnit` + `WebApplicationFactory<Program>`:
    - Teste de concorrência com 10 requisições simultâneas para o mesmo slot (garantir exatamente 1 sucesso e 9 conflitos 409).
    - Teste de tentativa de agendamento com coleta não validada (deve falhar).
    - Teste de quebra de perfil (atendente tentando ler evolução clínica -> 403).
    - Teste de IDOR (paciente tentando ler prontuário de outro paciente -> 403).
    - Teste do ciclo de compartilhamento e revogação.
    - Teste de idempotência na conclusão da consulta (não duplica convites/pesquisa).
- Documentação completa na pasta `docs/`:
    - `docs/arquitetura.md`: diagramas Mermaid, decisões técnicas, monólito modular.
    - `docs/permissoes.md`: matriz RBAC detalhada e regras de autorização de dados.
    - `docs/modelo-dados.md`: schemas do PostgreSQL e MongoDB.
    - `docs/fluxos.md`: fluxogramas de triagem, agendamento, atendimento e consentimento.
    - `README.md`: guia completo de execução passo a passo, variáveis de ambiente, seeds, reset de banco e roteiro para apresentação acadêmica.

7. Dados Seed para Demonstração

Para viabilizar uma apresentação imediata e sem atritos, o sistema virá pré-carregado com dois cenários:

1. **Cenário Histórico Concluído (Paciente 1 - "Carlos Silva")**:
    - Já realizou triagem e consulta prévia na *Clínica Central*.
    - Possui documento anexado (exame laboratorial fictício em PDF).
    - Possui autorização concedida para a *Clínica Esperança*.
    - Permite demonstrar imediatamente a visualização de prontuário cross-institution pelo Médico da Clínica Esperança.
2. **Cenário Novo para Execução ao Vivo (Paciente 2 - "Mariana Souza")**:
    - Conta pronta para executar o roteiro do zero durante a banca:
        1. Login como Mariana -> Início do Chat simulado (WhatsApp).

2. Resposta às perguntas guiadas -> Envio ao enfermeiro.
3. Login como Enfermeiro da Clínica Central -> Revisão da conversa, correção de alergia e Validação.
4. Retorno ao perfil de Mariana -> Escolha de slot livre com o Dr. João.
5. Login como Dr. João -> Atendimento, visualização do resumo validado, registro da evolução clínica e conclusão.
6. Notificação simulada na caixa de entrada de Mariana -> Resposta à pesquisa de satisfação.
7. Login como Atendente -> Verificação do Dashboard operacional atualizado em tempo real.
8. Demonstração de segurança: Atendente tentando chamar URL de evolução médica e recebendo `403 Forbidden`.

8. Critérios de Aceite e Verificação Final

Critério	Método de Verificação
Execução local sem serviços pagos	`docker-compose up -d` + `dotnet run` + `ng serve` sem internet externa.
Chat determinístico sem IA	Responde às 8 perguntas fixas, gera resumo por template, recusa entradas fora do script.
Agendamento condicionado	Backend rejeita agendamento se status da coleta != `Validada`.
Concorrência de agenda	Teste de carga/concorrência comprova zero sobreposições e retorno 409 consistente.
Isolamento clínico do atendente	Endpoint de histórico/evolução retorna 403 para token de Atendente.
Compartilhamento com revogação	Médico B perde acesso imediato aos dados de A assim que o paciente revoga o consentimento.
Pesquisa e notificações	Conclusão gera notificações e formulário de pesquisa uma única vez (idempotente).
Documentação e testes	100% dos testes de integração passando e documentação em `docs/` completa.